

Contents

1	Introduction	3
1.1	Objectives	3
2	Sending money using locks	4
Exercise 1.1		5
Actions		5
Processes		6
Exercise 1.2		6
Weak Bisimilarity		7
Exercise 1.3		10
Exercise 1.4		11
First approach		11
Second approach		12
3	Simulate dependent processes in a server	13
Exercise 2.1		14
Exercise 2.2		14
Exercise 2.3		16
Bad implementation		17
Busy-waiting implementation		17
Non-busy-waiting implementation		17

Chapter 1

Introduction

1.1 Objectives

1. .

Chapter 2

Sending money using locks

This chapter will focus on modeling in CCS the following scenario (and variants of it):

Figure 1 represents a simplified diagram where three participants use a `send€` action to represent an atomic transfer of money. This scenario does not include locks, which can lead to deadlocks when improperly implemented.

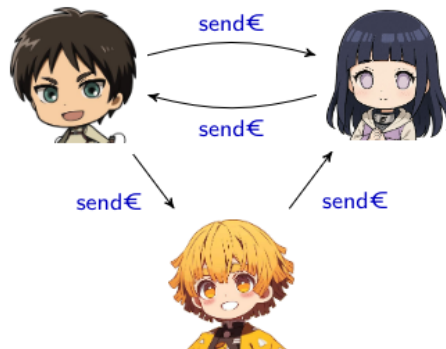


Figure 1: Example scenario where different participants send money to each other

Exercise 1.1

Firstly, a simplified version of the original scenario is considered:

- two participants always trying to send money to each other;
- the amount of money is ignored;
- `locks` are used in a naive way to force deadlocks to occur.

The subsequent model of the scenario under consideration is formulated using CCS, with the consideration of two participants, Zenitsu and Hinata. Figure 2 illustrates the respective LTS.

```
let
  HinToZenDL = acqLockHin'.acqLockZen'.sendMoney.
               relLockZen'.relLockHin'.HinToZenDL;

  ZenToHinDL = acqLockZen'.acqLockHin'.sendMoney.
               relLockHin'.relLockZen'.ZenToHinDL;

  LockHinDL = acqLockHin.relLockHin.LockHinDL;

  LockZenDL = acqLockZen.relLockZen.LockZenDL;

in
  (HinToZenDL | ZenToHinDL | LockHinDL | LockZenDL)
  \ {acqLockHin, acqLockZen, relLockHin, relLockZen}
```

Actions

- `acqLockHin`: acquisition of Hin's lock (sync);
- `relLockHin`: release of Hin's lock (sync);
- `acqLockZen`: acquisition of Zen's lock (sync);
- `relLockZen`: release of Zen's lock (sync);
- `sendMoney`: transfer of money.

It should be noted that each acquisition simulates Scala's `synchronized` construct behaviour in conjunction with its corresponding release. Moreover, it is not necessary to synchronise `sendMoney`, since the actions previously discussed ensure that this operation is performed atomically.

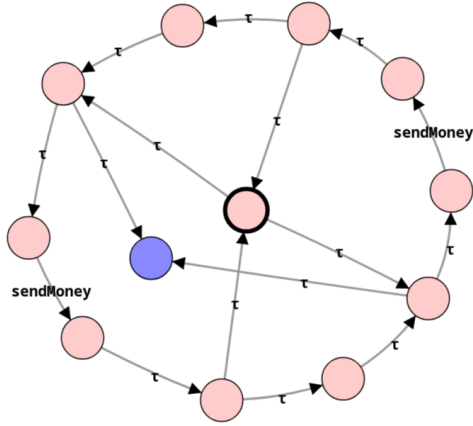


Figure 2: naive LTS

Processes

- `HinToZenDL`: representation of Hin trying to send money to Zen;
- `LockHinDL`: Hin's Lock associated with `HinToZenDL`;
- `ZenToHinDL`: representation of Zen trying to send money to Hin;
- `LockZenDL`: Zen's Lock associated with `ZenToHinDL`.

Exercise 1.2

The objective of this exercise is to adapt the model from the previous exercise to fix the deadlock. The approach used was to define a fixed order for lock acquisition. The idea is that Hin is always the first to acquire the lock, irrespective of his role, whether as the sender or the receiver.

let

`HinZenSend = acqLockHin'.acqLockZen'.sendMoney.`

```

    relLockZen'.relLockHin'.HinZenSend;

    LockHin = acqLockHin.relLockHin.LockHin;

    LockZen = acqLockZen.relLockZen.LockZen;

in
  (HinZenSend | HinZenSend | LockHin | LockZen)
  \ {acqLockHin, acqLockZen, relLockHin, relLockZen}

```

To date, two systems have been implemented in different ways to achieve the same desired outcome, namely the ability for two participants to transfer money to each other. These two systems are **not** weakly bisimilar, and we can prove it by simulating an attacker-defender game. The idea behind this is that the naive model exhibits a deadlock state, which the attacker can always access, irrespective of the defender's actions. This assertion is predicated on the notion that, at a certain point, the defender's capacity to replicate the attacker's actions becomes exhausted.

Weak Bisimilarity

State 1

A: strong τ (LTS1) **D:** weak τ (LTS2)

LTS1: $(acqLockZen!.sendMoney.relLockZen!.relLockHin!.HinToZenDL$
 $| ZenToHinDL | relLockHin?.LockHinDL | LockZenDL)$
 $\setminus \{acqLockHin, acqLockZen, relLockHin, relLockZen\}$

LTS2: $(acqLockZen!.sendMoney.relLockHin!.relLockZen!.HinZenSend$
 $| HinZenSend | relLockHin?.LockHin | LockZen)$
 $\setminus \{acqLockHin, acqLockZen, relLockHin, relLockZen\}$

State 2

A: strong τ (LTS1) **D:** weak τ (LTS2)

LTS1: (*acqLockZen!.sendMoney.relLockZen!.relLockHin!.HinToZenDL*
 | *acqLockHin!.sendMoney.relLockHin!.relLockZen!.ZenToHinDL*
 | *relLockHin?.LockHinDL* | *relLockZen?.LockZenDL*)
 \ {*acqLockHin*, *acqLockZen*, *relLockHin*, *relLockZen*}

LTS2: (*sendMoney.relLockHin!.relLockZen!.HinZenSend*
 | *HinZenSend* | *relLockHin?.LockHin* | *relLockZen?.LockZen*)
 \ {*acqLockHin*, *acqLockZen*, *relLockHin*, *relLockZen*}

State 3

A: strong *sendMoney* (LTS2) **D:** Impossible (LTS1)

LTS1: $\emptyset$

LTS2: (*relLockHin!.relLockZen!.HinZenSend*
 | *HinZenSend* | *relLockHin?.LockHin* | *relLockZen?.LockZen*)
 \ {*acqLockHin*, *acqLockZen*, *relLockHin*, *relLockZen*}

The defender was unable to replicate the action of the attacker. Consequently, we retrace the defender's actions to the previous state where alternative responses were possible (i.e. State 2).

State 4

A: strong τ (LTS1) **D:** weak τ (LTS2)

LTS1: (*acqLockZen!.sendMoney.relLockZen!.relLockHin!.HinToZenDL*
 | *acqLockHin!.sendMoney.relLockHin!.relLockZen!.ZenToHinDL*
 | *relLockHin?.LockHinDL* | *relLockZen?.LockZenDL*)
 \ {*acqLockHin*, *acqLockZen*, *relLockHin*, *relLockZen*}

LTS2: (*acqLockZen!.sendMoney.relLockHin!.relLockZen!.HinZenSend*
 | *HinZenSend* | *relLockHin?.LockHin* | *LockZen*)
 \ {*acqLockHin*, *acqLockZen*, *relLockHin*, *relLockZen*}

State 5

A: strong τ (LTS2) **D:** weak τ (LTS1)

LTS1: (*acqLockZen!.sendMoney.relLockZen!.relLockHin!.HinToZenDL*
 | *acqLockHin!.sendMoney.relLockHin!.relLockZen!.ZenToHinDL*
 | *relLockHin?.LockHinDL* | *relLockZen?.LockZenDL*)
 \ {*acqLockHin*, *acqLockZen*, *relLockHin*, *relLockZen*}

LTS2: (*sendMoney.relLockHin!.relLockZen!.HinZenSend*
 | *HinZenSend* | *relLockHin?.LockHin* | *relLockZen?.LockZen*)
 \ {*acqLockHin*, *acqLockZen*, *relLockHin*, *relLockZen*}

This state identical to State 2, which implies that the defender loses the game. Consequently, we retrace the defender's actions to the previous state where alternative responses were possible (i.e. State 1).

State 6

A: strong τ (LTS1) **D:** weak τ (LTS2)

LTS1: (*acqLockZen!.sendMoney.relLockZen!.relLockHin!.HinToZenDL*
 | *ZenToHinDL* | *relLockHin?.LockHinDL* | *LockZenDL*)
 \ {*acqLockHin*, *acqLockZen*, *relLockHin*, *relLockZen*}

LTS2: (*HinZenSend* | *HinZenSend* | *LockHin* | *LockZen*)
 \ {*acqLockHin*, *acqLockZen*, *relLockHin*, *relLockZen*}

State 7

A: strong τ (LTS1) **D:** weak τ (LTS2)

LTS1: (*acqLockZen!.sendMoney.relLockZen!.relLockHin!.HinToZenDL*
 | *acqLockHin!.sendMoney.relLockHin!.relLockZen!.ZenToHinDL*
 | *relLockHin?.LockHinDL* | *relLockZen?.LockZenDL*)
 \ {*acqLockHin*, *acqLockZen*, *relLockHin*, *relLockZen*}

LTS2: (*acqLockZen!.sendMoney.relLockHin!.relLockZen!.HinZenSend*
 | *HinZenSend* | *relLockHin?.LockHin* | *LockZen*)
 \ {*acqLockHin*, *acqLockZen*, *relLockHin*, *relLockZen*}

This state identical to State 4, which implies that the defender loses the game. Consequently, we retrace the defender's actions to the previous state where alternative responses were possible (i.e. State 7).

State 8**A:** strong τ (LTS1) **D:** weak τ (LTS2)
$$\begin{aligned}
LTS1: & (acqLockZen!.sendMoney.relLockZen!.relLockHin!.HinToZenDL \\
& \mid acqLockHin!.sendMoney.relLockHin!.relLockZen!.ZenToHinDL \\
& \mid relLockHin?.LockHinDL \mid relLockZen?.LockZenDL) \\
& \setminus \{acqLockHin, acqLockZen, relLockHin, relLockZen\}
\end{aligned}$$

$$\begin{aligned}
LTS2: & (HinZenSend \mid HinZenSend \mid LockHin \mid LockZen) \\
& \setminus \{acqLockHin, acqLockZen, relLockHin, relLockZen\}
\end{aligned}$$
State 9**A:** strong τ (LTS2) **D:** weak τ (LTS1)
$$\begin{aligned}
LTS1: & (acqLockZen!.sendMoney.relLockZen!.relLockHin!.HinToZenDL \\
& \mid acqLockHin!.sendMoney.relLockHin!.relLockZen!.ZenToHinDL \\
& \mid relLockHin?.LockHinDL \mid relLockZen?.LockZenDL) \\
& \setminus \{acqLockHin, acqLockZen, relLockHin, relLockZen\}
\end{aligned}$$

$$\begin{aligned}
LTS2: & (acqLockZen!.sendMoney.relLockHin!.relLockZen!.HinZenSend \\
& \mid HinZenSend \mid relLockHin?.LockHin \mid LockZen) \\
& \setminus \{acqLockHin, acqLockZen, relLockHin, relLockZen\}
\end{aligned}$$

This state identical to State 4, which implies that the defender loses the game. It would be possible to backtrack again by selecting the process that has not been altered, *HinZenSend*. Nevertheless, it is evident that the outcome would be the same.

Exercise 1.3

In this exercise, the scenario from Figure 1 is employed, utilising locks in a naive way (where deadlocks may occur). The approach adopted in this instance is equivalent to that utilised in Exercise 1.1, but for the full scenario.

```

let
  HinToErenDL = acqLockHin'.acqLockEren'.sendMoney.
                relLockEren'.relLockHin'.HinToErenDL;

  ErenToHinDL = acqLockEren'.acqLockHin'.sendMoney.

```

```

        relLockHin'.relLockEren'.ErenToHinDL;

ErenToZenDL = acqLockEren'.acqLockZen'.sendMoney.
              relLockZen'.relLockEren'.ErenToZenDL;

ZenToHinDL = acqLockZen'.acqLockHin'.sendMoney.
              relLockHin'.relLockZen'.ZenToHinDL;

LockHinDL = acqLockHin.relLockHin.LockHinDL;

LockErenDL = acqLockEren.relLockEren.LockErenDL;

LockZenDL = acqLockZen.relLockZen.LockZenDL;

in
  (HinToErenDL | ErenToHinDL | ErenToZenDL | ZenToHinDL
   | LockHinDL | LockErenDL | LockZenDL)
  \
  {acqLockHin, acqLockZen, acqLockEren, relLockHin,
   relLockZen, relLockEren}

```

Exercise 1.4

The scenario is now being generalised through the implementation of value-passing CCS, with three equivalent participants who can always send messages to any other participant. Recall the exercise 1.2 where fixed order for lock acquisition was used in order not to have deadlocks. This exercise uses the same approach, where the participant with the lower identifier, must acquire the lock first.

First approach

As previously stated in Exercise 1.2, the fixed order for lock acquisition was utilised in order to prevent deadlocks. This approach employs the same methodology, where the participant with the lower identifier must acquire the lock first. However, given the the generalisation of the model, all locks communicate through the same channel. Thus, when a participant attempts to acquire a lock, the action may synchronise with any lock process rather than exclusively with the lock associated with the participant. If it happens that it does not synchronise with the participant's lock, the possibility of deadlock and inconsistency arising is increased.

```

FromTo[x, y] := (when (x < y) acqLock!x.acqLock!y.sendMoney.
                  relLock!y.relLock!x.FromTo[x, y])
                +
                (when (x >= y) acqLock!y.acqLock!x.sendMoney.
                  relLock!x.relLock!y.FromTo[x, y])

Lock[x] := acqLock?x.relLock?x.Lock[x]

(FromTo[1, 2] | FromTo[2, 1] | FromTo[1, 3] | FromTo[3, 1] |
 FromTo[2, 3] | FromTo[3, 2] | Lock[1] | Lock[2] | Lock[3])
\ {acqLock, relLock}

```

Second approach

*****FALAR AQUI*****

```

FromTo[x, y] := (when (x < y) acqLock!x.ackAcqLock?z : Int.
                  (when (x == z) acqLock!y.ackAcqLock?z : Int.
                    (when (y == z) sendMoney.relLock!y.ackRelLock?z : Int.
                      (when (y == z) relLock!x.ackRelLock?z : Int.
                        (when (x == z) FromTo[x, y])))))
                +
                (when (x >= y) acqLock!y.ackAcqLock?z : Int.
                  (when (y == z) acqLock!x.ackAcqLock?z : Int.
                    (when (x == z) sendMoney.relLock!x.ackRelLock?z : Int.
                      (when (x == z) relLock!y.ackRelLock?z : Int.
                        (when (y == z) FromTo[x, y])))))

Lock[x] := acqLock?z.(when (x == z) ackAcqLock!x.relLock?z.
                      (when (x == z) ackRelLock!x.Lock[x]))

(FromTo[1, 2] | FromTo[2, 1] | FromTo[1, 3] | FromTo[3, 1] |
 FromTo[2, 3] | FromTo[3, 2] | Lock[1] | Lock[2] | Lock[3])
\ {acqLock, relLock}

```

Chapter 3

Simulate dependent processes in a server

This chapter will focus on adding support to the execution of simple instructions in a working skeleton server, following some requirements as listed below:

1. the pending instructions should be managed by a thread pool with a fixed number of threads;
2. the code that is executed (concurrently) when receiving a request should be as small as possible;
3. if an instruction is ready to execute and there is an unused thread, the instruction should be executed as soon as possible;
4. shared data structures should be used to model the state of the server;
5. three different kinds of requests can be made to the server:
 - run a given block of instructions;
 - ask for the state of the server;
 - reset the state of the server.
6. the result of the printing instruction should be stored by the server in its internal state (sent to the client when requested).

Exercise 2.1

It is important to note that the initial counter should count the quantity of requests that have been received. However it is not thread-safe (it may provide unexpected results). As illustrated in Figure 3, this is an example of the kind of unanticipated outcomes that could be obtained.



Figure 3: Example input where unexpected results are evident

It is to be expected that the counter will register four, given that four requests are being sent to the server. However, the result was not 4, but 3. This was due to the fact that the counter variable was not thread-safe. As demonstrated in Figure 4, this sequence of events can be understood as a potential execution order that gave rise to the observed behaviour, where both threads write the value 1 to the counter. The solution to the issue lies in ensuring that the counter variable is made thread-safe. One method of achieving this is by changing its type from `Int` to `AtomicInteger`, and using its atomic operations (i.e. `incrementAndGet`).

Exercise 2.2

The next step is to extend the server to run the required instructions, ignoring for now the `after` clause. In order to achieve this, a thread pool with a predetermined number of threads (workers) was implemented so that the instructions could be executed concurrently. Listing 1 presents the implementation of a worker. Lock-free programming was also employed for the purpose of storing the result of print instructions, by using an atomic variable, `AtomicReference[List[String]]`.

It is important to acknowledge that, if the shared structure was non-thread-safe, a comparable issue to that observed in exercise 2.1 with the counter would be encountered. In other words, one update could be overwritten by another,

```

class Worker(
  tasks: mutable.Queue[(String, Option[Int])],
  content: AtomicReference[List[String]]
) extends Thread {
  setDaemon(true)
  def poll() = tasks.synchronized {
    while (tasks.isEmpty) tasks.wait()
    tasks.dequeue()
  }

  override def run() = while (true) {
    val (str, delayOpt) = poll()
    var delay = delayOpt match {
      case Some(d) => d
      case None    => 0
    }
    Thread.sleep(delay * 1000)

    val formatter = DateTimeFormatter.ofPattern("yyyy/MM/dd HH:mm:ss")
    val element =
      "[" ++ LocalDateTime.now().format(formatter) ++ "]" ++ str
    content.updateAndGet { old =>
      old :+ element
    }
  }
}

```

Listing 1: Worker implementation

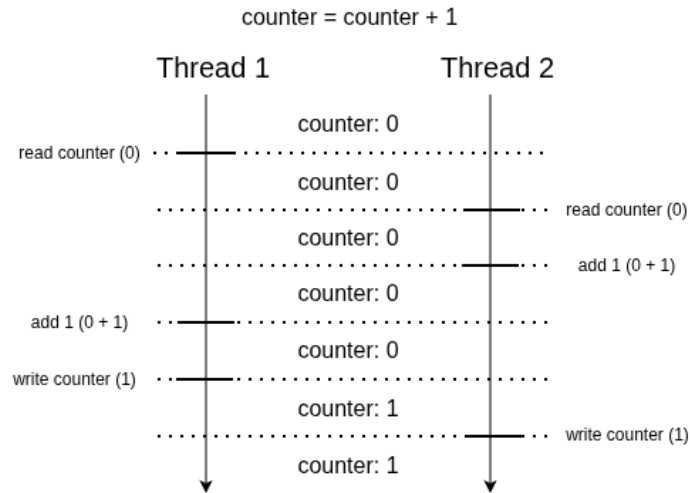


Figure 4: Example of a bad sequence of events

causing inconsistency. Figure 5 illustrates an example where such an inconsistency could occur. The first instruction could read the original value, [], and compute an updated version containing "hello1". However, prior to the rewriting of the update to the shared variable, it is possible that the second instruction would also read the same old value, [], and compute a different update containing "hello2". Consequently, the second write has the potential to overwrite the first one, resulting in the loss of one of the messages, and therefore fewer print messages.

```

Input program
1 # comment
2 server: localhost:8080
3 print "hello1"
4 print "hello2"
5 print "hello3"
6 print "hello4"

```

Figure 5: Example input where unexpected results are evident

Exercise 2.3

Recall that the **after** clause was left unimplemented. In this exercise, three different approaches are examined and then compared with each other.

Bad implementation

The first idea was to create a data structure representing the topological ordering of the instruction dependencies. Therefore, two shared variables were introduced. The first of these variables was a map that mapped an instruction identifier to the instructions that depended on that key instruction to finish. The second variable was another map that also mapped an instruction identifier to an Int value representing the number of instructions that it was still contingent on. At first, all the instructions without any dependencies, are added to the queue. After the execution of an instruction, all instructions depending on it were updated by decreasing their current dependency count by one. Upon the occurrence of a count of zero for the designated instruction, the instruction was consequently added to the queue of tasks.

However, given the utilisation of shared variables among threads, the identification of the instruction number as the sole identifier is not enough. Consequently, the atomic counter was utilised as a prefix. The chosen identifier is where the problem resides, as there is a possibility of the counter being reset and two different requests using the same counter on the instruction identifier.

Busy-waiting implementation

The initial concept was not entirely unsuccessful. However the proposal of distributing the maps amongst the threads was not a viable proposition. The solution to this issue was to implement the use of unique maps for each individual request. These maps were then provided as arguments to the threads, with the intention that they be updated atomically. The verification of whether an instruction possessed counter 0, a prerequisite for its processing, was conducted in a while loop in the runProcess() method. Whilst the proposed solution is indeed correct and consistent, it is important to note that busy waiting occurs. This is a process whereby the loop repetitively verifies a condition, thereby requiring processor power unnecessarily.

Non-busy-waiting implementation

The final approach, was built on top of the previous one, however, with only one difference. Instead of repetitively checking for 0, we only check if a counter is 0, when it is updated after the execution of an instruction.

Chapter 4

Conclusion